Chris Young

Logan Douglas

Nathan Howell

SLO6 Experimentation and Analyze

Purpose

A lot of our experimentation was trying our data against a neural network and seeing how it worked. We needed to know if this data was good enough to support the feature in our app, and so this experimentation was necessary for the development of our app.

The main purpose for the experimentation was to discover how the feature would look and feel. For one, we weren't sure how many data points we wanted to predict and show our user. We also weren't sure how we wanted to display the information. Would we give the user just close price predictions or full candlestick predictions? We needed to discover these things for ourselves with experiments.

Our secondary purpose was to see how good of a model we could get. We made a lot of the same choices here to improve model performance. Some tradeoffs would have to be made between model performance and the actual displaying of the information, but we could only make these decisions by experimenting with the model.

Experiment Setup

Back when we were using Yahoo Finance for our historical data, we had Claude make a quick program for training and testing a neural network. This was also when we

were doing more of the trend analysis and not as much price predictions. After training and testing the model, we had it display the results in a graph.

The graph showed the sample data for the trend we were trying to match and a few of the trends it found. We also had it score how confident it was that it matched the pattern. It wasn't a great model, but it was only the first trial and it also served as a good demo for presenting our idea for an Epic. We also got to see a lot of the stock data ourselves when working on our app and we rarely ever saw these trends, which showed they weren't very common and they wouldn't have given us a good generalized model.

Earlier this semester, we had experimented with a prediction model on two years worth of stock data for AAPL. For this experiment, we used walk-forward validation, where the model trains, tests, and validates using a sliding window for the last two years. We graphed the results comparing the models predictions to the actual stock data, and it was very accurate, but was also overfit since it was only using AAPL's data.

For our next trials, we experimented with multiple stocks. We first started small with a group of one-hundred stocks, but we expanded after this with bigger groups until we started testing and training with all 12,000 stocks stored in our database. We even experimented with changing the inputs. We add a symbol input which is just the ticker symbol transformed into an embedding (a vector) so that the model would be able to make predictions based on the stock and the lookback window. This also overfitted our model, but we wouldn't have known had we not tried it.

We eventually changed the way we were training, testing, and validating the model and instead of training with one stock at a time, we trained, tested, and validated

with multiple stocks at a time. Of course, this was very memory inefficient so we needed to do it in batches. We chose to do batches of 500 stocks at a time.

We calculated the MAPE (Mean Absolute Percentage Error) for each session of training and testing to analyze how the model fitted. We also had it compute an overall accuracy after training and testing was finished. The MAPE is a score of how off the predicted points were, which would tell us how each phase of training and testing went. If we had a ridiculous MAPE, then we would know that the model was overfit.

For example, If one batch had an MAPE of 1% and the next had an MAPE of 15%, then we would know that the model was overfit to the previous batch of data. The overall accuracy score was to show how many of the data points were predicted correctly during training/testing to give us an idea of how good the model was.

Conclusions

After further experimentation with the pattern matching model, we felt like we needed to move away from trend analysis and generalize our model more for all stocks. A lot of these patterns that you see in stocks aren't common so you don't see them in every stock's history. The model we created for pattern matching gave us very low confidence scores because the closest patterns it could find were nowhere near the actual pattern.

Looking at the stock data ourselves also helped us confirm that this wasn't a great method for doing this. The amount of times we have seen a double top, for example, in our stock data is very low. Because of this, we decided to go with a more generalized prediction feature that shows users what the stock price might look like in

the next few hours. Of course, we wouldn't have known to do this if we didn't experiment with it.

These experiments changed our feature designs and helped shape our decisions for how we wanted those features to work. At first, we wanted to provide recommended options to users based on trends we say in the stock historical data. While this was a very good educational feature for users, it was not generalized enough for all of the stocks. Our users also might not understand what to infer from the trends and so it wouldn't have been user friendly for all of our target audience.

This also improved our business understanding of the app. Our idea for the trend analysis and predictions were a lot different before our experiments. We were able to learn that the further ahead in time we looked, the less plausible our predictions would be, and that doesn't help a user. A small period of time like 3 hours is a little more likely and the forecast was only to give the users a glimpse into what could happen to the price.